

FocusVolution

Aplicação Android para gestão de sessões de foco com gamificação, autenticação segura e administração de utilizadores.

Projeto desenvolvido no âmbito da PAP (Prova de Aptidão Profissional) do curso **Técnico de Gestão e Programação de Sistemas de Informação (GPSI)**.

Índice

- [Descrição do Projeto](#)
 - [Tecnologias Utilizadas](#)
 - [Arquitetura da Aplicação](#)
 - [Requisitos de Sistema](#)
 - [Estrutura do Projeto](#)
 - [Estrutura do Código](#)
 - [Base de Dados](#)
 - [Mecânicas de Gamificação](#)
 - [Segurança](#)
 - [Animações](#)
 - [Navegação](#)
 - [Configuração](#)
 - [Instalação e Execução](#)
 - [Gerar APK](#)
 - [Funcionalidades](#)
 - [Credenciais de Teste](#)
 - [Utilização da Aplicação](#)
 - [Resolução de Problemas](#)
-

Descrição do Projeto

FocusVolution é um temporizador de foco que incentiva o utilizador a completar sessões de concentração através de um sistema de níveis (gamificação). Cada 5 sessões concluídas sobem um nível (máximo 10). Inclui registo com verificação por email, histórico de sessões com tags, gráfico de produtividade semanal, perfil de utilizador, definições personalizáveis e painel de administração.

Principais funcionalidades:

- Temporizador de contagem decrescente com execução em foreground
 - Sistema de níveis (1 a 10) com imagens cerebrais distintas
 - Registo com verificação por email (SMTP Gmail + código 6 dígitos)
 - Histórico de sessões com filtro por tags
 - Gráfico de produtividade semanal
 - Perfil de utilizador com edição de username/password
 - Definições personalizáveis (duração padrão, som, lembrar último valor)
 - Painel de administração (listar, ver histórico, eliminar utilizadores)
 - Animações: pop-ups, transições entre ecrãs, pulsação do timer, stagger list, etc.
-

Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Kotlin	1.9.24	Linguagem principal
Jetpack Compose	BOM 2024.06	UI declarativa (todos os ecrãs)
Material 3	(BOM)	Design system moderno
Room	2.6.1	Base de dados local (SQLite) — 3 tabelas, 5 migrações
Navigation Compose	2.7.7	Navegação entre ecrãs com transições animadas
Lifecycle ViewModel Compose	2.8.4	ViewModel integrado com Compose
Coroutines	1.8.1	Operações assíncronas e Flow
JavaMail (Android)	1.6.6	Envio de emails de verificação via SMTP Gmail
Android SDK	34 (compile), 24 (min)	SDK alvo
Gradle + KSP	8.x / 1.9.24-1.0.20	Sistema de build e processamento de anotações Room

Arquitetura: MVVM com injeção manual de dependências (sem Hilt/Dagger/Koin).

Arquitetura da Aplicação

Padrão MVVM

A aplicação segue o padrão **Model-View-ViewModel**:

- **Model (Room + Repository):** Base de dados SQLite local gerida pelo Room. O `FocusVolutionRepository` centraliza toda a lógica de negócios (registo de sessões, cálculo de nível, autenticação).
- **ViewModel (MainViewModel):** Mantém o estado da UI principal (timer, nível, sessões) e expõe `StateFlow` para o Compose observar reativamente.
- **View (Compose Screens):** Todos os ecrãs são componentes Compose que reagem a `StateFlow` / `Flow` e emitem eventos para o ViewModel/Repository.

Service Locator

A `FocusVolutionApp` (Application class) funciona como **service locator**, expondo instâncias `lazy` de `AppDatabase`, `SettingsManager` e `FocusVolutionRepository` para toda a aplicação.

Não é usado Hilt, Dagger ou Koin por simplicidade e para reduzir a complexidade do build.

Ciclo de Vida do Timer

O temporizador corre num **Foreground Service** (`TimerForegroundService`) para continuar a contar mesmo quando a aplicação está em segundo plano. A comunicação entre o Service e o ViewModel é feita através de um singleton (`TimerServiceStateStore`) que expõe o estado atual do timer como `StateFlow`.

```

MainScreen (Compose)
|  observa StateFlow do TimerServiceStateStore
|  envia comandos via bindService / startService
▼
TimerForegroundService
|  atualiza TimerServiceStateStore em cada tick
|  publica notificação com tempo restante
▼
TimerServiceStateStore (singleton)
|  StateFlow<TimerServiceState>
|
├─ MainViewModel (lê o estado)
└─ MainScreen (UI reage)

```

Requisitos de Sistema

Para Desenvolvimento

- **Android Studio** Hedgehog (2023.1.1) ou superior
- **JDK 17** (incluído no Android Studio)
- **Git** para clonar o repositório
- **Android SDK 34** (instalado via Android Studio SDK Manager)

Para Execução (dispositivo)

- **Android 7.0** (API 24) ou superior
- **Espaço:** ~50 MB livres
- **Conexão à Internet** (apenas para registo/verificação de email)
- **Permissões:** Notificações (Android 13+)

Estrutura do Projeto

```

app/
├─ src/main/
│   └─ java/com/focusvolution/app/
│       ├── FocusVolutionApp.kt      # Application class (service locator)
│       ├── MainActivity.kt          # Activity principal
│       ├── SettingsManager.kt       # SharedPreferences (definições)
│       └─ config/
│           └─ AppConfig.kt          # Credenciais admin centralizadas
│       └─ data/
│           ├── local/
│               ├── AppDao.kt        # DAO (queries Room para sessões)
│               ├── UserDao.kt       # DAO (queries Room para utilizadores)
│               ├── AppDatabase.kt   # Base de dados (versão 6)
│               ├── SessionEntity.kt # Entidade sessões (com tag)
│               ├── UserEntity.kt    # Entidade utilizadores
│               └─ AppStateEntity.kt # Entidade estado global
│           └─ repository/
│               └─ FocusVolutionRepository.kt # Lógica de negócios
│       └─ email/

```

```

├── EmailConfig.kt          # Configuração SMTP
├── EmailService.kt        # Envio de emails (JavaMail)
├── service/
│   ├── TimerForegroundService.kt # Serviço foreground do timer
│   ├── TimerServiceState.kt     # Estado do temporizador
│   └── TimerServiceStateStore.kt # Singleton (ponte Service -> ViewModel)
├── ui/
│   ├── components/
│   │   ├── AppCharacter.kt      # Imagem do cérebro por nível (Crossfade)
│   │   ├── LevelUpPopup.kt     # Pop-up subida de nível
│   │   └── LevelDownPopup.kt   # Pop-up descida de nível
│   ├── main/
│   │   ├── MainUiState.kt      # Estado da UI principal
│   │   └── MainViewModel.kt    # ViewModel principal
│   ├── navigation/
│   │   └── AppNavigation.kt     # Rotas e navegação + transições
│   ├── screens/
│   │   ├── LoginScreen.kt      # Login (email/username + password)
│   │   ├── RegisterScreen.kt   # Registo com validação
│   │   ├── VerifyCodeScreen.kt # Verificação de email (6 dígitos)
│   │   ├── MainScreen.kt       # Temporizador + nível + progresso
│   │   ├── SettingsScreen.kt   # Definições (duração, som, etc.)
│   │   ├── ProfileScreen.kt    # Perfil (username, password, total foco)
│   │   ├── ChartScreen.kt      # Gráfico de barras semanal (Canvas)
│   │   ├── SessionHistoryScreen.kt # Histórico pessoal com filtro tags
│   │   ├── AdminScreen.kt      # Paineil admin (listar/eliminar users)
│   │   └── AdminUserHistoryScreen.kt # Histórico de outro user (admin)
│   └── theme/
│       └── Theme.kt             # Cores Material 3 (dark/light)
├── res/
│   ├── drawable/              # cerebro_1.png a cerebro_10.png
│   └── ...                     # Outros recursos
├── AndroidManifest.xml
├── build.gradle.kts
└── settings.gradle.kts

```

Estrutura do Código

Principais Classes e Responsabilidades

Ficheiro	Responsabilidade
FocusVolutionApp.kt	Application class; service locator (DB, SettingsManager, Repository)
MainActivity.kt	Single Activity; configura tema, inicia navegação e foreground service
SettingsManager.kt	Abstração sobre SharedPreferences (duração padrão, som, remember)
FocusVolutionRepository.kt	Camada de repositório: regista sessões, calcula níveis, autenticação, verificação email, gestão de registos pendentes

AppDatabase.kt	Room database com 5 migrações e método <code>getDatabase()</code> thread-safe (double-checked locking)
AppDao.kt	DAO para <code>sessions</code> e <code>app_state</code> : queries de insert, delete, observe (Flow)
UserDao.kt	DAO para <code>users</code> : CRUD completo, verificação de unicidade, atualização de stats
TimerForegroundService.kt	Foreground service que executa o timer em background com notificação persistente
TimerServiceStateStore.kt	Singleton que expõe <code>StateFlow<TimerServiceState></code> para comunicação Service ↔ ViewModel
MainViewModel.kt	ViewModel principal: estado do timer (horas, minutos, segundos), interação com service
AppNavigation.kt	NavHost Compose com todas as rotas e transições animadas
EmailService.kt	Envio de emails SMTP via JavaMail com template HTML
Theme.kt	Definição de cores Material 3, dark/light theme
AppCharacter.kt	Componente <code>Crossfade</code> que mostra a imagem cerebral correspondente ao nível
LevelUpPopup.kt	Pop-up animado de subida de nível com bounce
LevelDownPopup.kt	Pop-up animado de descida de nível

Fluxo de Dados (Data Flow)

Timer:

```
User Input (Compose) → MainScreen → MainViewModel → TimerForegroundService
  → TimerServiceStateStore (StateFlow) → MainScreen (UI reage)
  → Ao terminar: vibra, regista sessão via Repository
```

Registo:

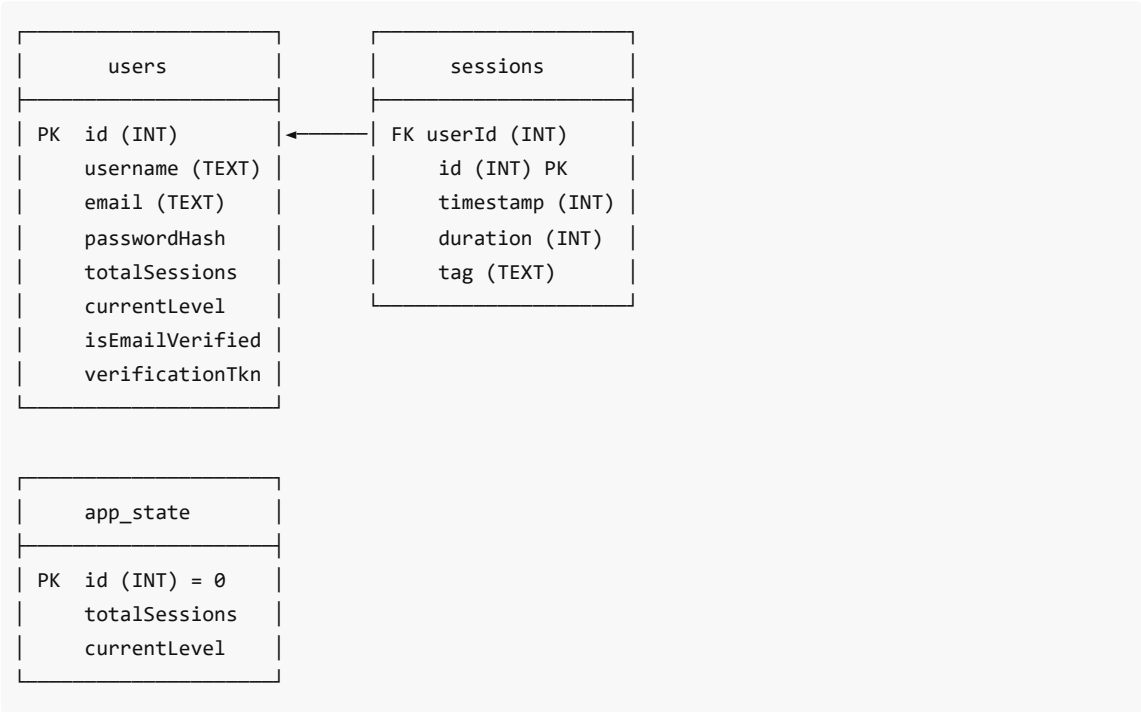
```
RegisterScreen → repository.register()
  └─ Valida campos (não vazio, email válido, password ≥ 6 chars)
  └─ Verifica duplicados (email + username)
  └─ Gera token UUID + código 6 dígitos
  └─ Guarda em SharedPreferences (pending_registrations)
  └─ Envia email via EmailService

VerifyCodeScreen → repository.verifyEmailWithCode()
  └─ Procura pending por código
  └─ Verifica expiração (24h)
  └─ Cria UserEntity na BD com isEmailVerified = true
  └─ Remove pending registration
```

Base de Dados

A aplicação utiliza **Room** (SQLite) como base de dados local. A base de dados está atualmente na **versão 6**, com 5 migrações acumuladas desde a versão 1.

Diagrama Entidade-Relação



SQL DDL (Criação das Tabelas)

```
-- Tabela: users
CREATE TABLE IF NOT EXISTS `users` (
    `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    `username` TEXT NOT NULL,
    `email` TEXT NOT NULL,
    `passwordHash` TEXT NOT NULL,
    `totalSessions` INTEGER NOT NULL DEFAULT 0,
    `currentLevel` INTEGER NOT NULL DEFAULT 1,
    `isEmailVerified` INTEGER NOT NULL DEFAULT 0,
    `verificationToken` TEXT DEFAULT NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS `index_users_email` ON `users` (`email`);
CREATE UNIQUE INDEX IF NOT EXISTS `index_users_username` ON `users` (`username`);
CREATE UNIQUE INDEX IF NOT EXISTS `index_users_verificationToken` ON `users`
(`verificationToken`);

-- Tabela: sessions
CREATE TABLE IF NOT EXISTS `sessions` (
    `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    `userId` INTEGER NOT NULL DEFAULT -1,
    `timestamp` INTEGER NOT NULL,
    `duration` INTEGER NOT NULL,
```

```
    `tag`          TEXT DEFAULT NULL
);

-- Tabela: app_state
CREATE TABLE IF NOT EXISTS `app_state` (
  `id`            INTEGER PRIMARY KEY NOT NULL DEFAULT 0,
  `totalSessions` INTEGER NOT NULL,
  `currentLevel`  INTEGER NOT NULL
);
```

Tabelas

Tabela users — armazena os utilizadores registados (incluindo conta admin). A password é guardada como hash SHA-256.

Coluna	Tipo	Descrição
id	INTEGER (PK)	Identificador único
username	TEXT (UNIQUE)	Nome de utilizador
email	TEXT (UNIQUE)	Email
passwordHash	TEXT	Hash SHA-256 da password
totalSessions	INTEGER	Total de sessões concluídas
currentLevel	INTEGER	Nível atual (1-10)
isEmailVerified	INTEGER (0/1)	Email verificado?
verificationToken	TEXT (NULLABLE, UNIQUE)	Token de verificação

Tabela sessions — cada linha representa uma sessão de foco concluída. A coluna `tag` permite categorizar sessões.

Coluna	Tipo	Descrição
id	INTEGER (PK)	Identificador único
userId	INTEGER	ID do utilizador
timestamp	INTEGER	Data/hora em epoch millis
duration	INTEGER	Duração em segundos
tag	TEXT (NULLABLE)	Tag opcional (ex: "Estudo", "Trabalho")

Tabela app_state — estado global usado apenas em modo convidado (userId = -1). Contém sempre uma única linha com id = 0.

Coluna	Tipo	Descrição
id	INTEGER (PK)	Sempre 0 (única linha)

totalSessions	INTEGER	Total de sessões (modo convidado)
currentLevel	INTEGER	Nível atual (modo convidado)

Migrações

Versão	Alteração
1 → 2	Criação da tabela users com id, username, email, passwordHash + índices únicos
2 → 3	Adiciona coluna userId à tabela sessions
3 → 4	Adiciona colunas totalSessions e currentLevel à tabela users
4 → 5	Adiciona isEmailVerified e verificationToken à tabela users + índice único
5 → 6	Adiciona coluna tag à tabela sessions

O ficheiro `schema.sql` na raiz do projeto contém o DDL completo para recriar a base de dados.

Mecânicas de Gamificação

Sistema de Níveis

O nível do utilizador é calculado pela fórmula:

```
nivel = min((totalSessoes / 5) + 1, 10)
```

Sessões Concluídas	Nível	Imagem
0-4	Nível 1	cerebro_1.png
5-9	Nível 2	cerebro_2.png
10-14	Nível 3	cerebro_3.png
15-19	Nível 4	cerebro_4.png
20-24	Nível 5	cerebro_5.png
25-29	Nível 6	cerebro_6.png
30-34	Nível 7	cerebro_7.png
35-39	Nível 8	cerebro_8.png
40-44	Nível 9	cerebro_9.png
45+	Nível 10	cerebro_10.png

Progressão

- **Subida de nível:** A cada 5 sessões concluídas, o nível sobe. Aparece um pop-up animado (`LevelUpPopup`) com bounce.

- **Queda de nível:** Sempre que o utilizador sai da aplicação 3 vezes durante uma sessão ativa (falha), desce 1 nível. Aparece um pop-up de apoio (`LevelDownPopup`).
- **Barra de progresso:** Mostra visualmente o tempo restante. Nos últimos 5 segundos, o texto do timer e a barra pulsam entre a cor primária e vermelho.

Tags (Categorização)

O utilizador pode associar uma tag textual a cada sessão (ex: "Estudo", "Trabalho", "Leitura"). As tags permitem filtrar o histórico de sessões e analisar a produtividade por categoria.

Gráfico Semanal

No ecrã de perfil, o utilizador pode aceder a um gráfico de barras desenhado com **Canvas** que mostra os minutos de foco acumulados por dia da semana (Domingo a Sábado), permitindo visualizar padrões de produtividade.

Segurança

Palavras-passe

As passwords são armazenadas como **hash SHA-256** antes de serem guardadas na base de dados. A hash é gerada no `FocusVolutionRepository.hashPassword()` :

```
private fun hashPassword(password: String): String {
    val digest = MessageDigest.getInstance("SHA-256")
    val hashBytes = digest.digest(password.toByteArray(Charsets.UTF_8))
    return hashBytes.joinToString("") { "%02x".format(it) }
}
```

Verificação de Email

O processo de registo é de **dois passos**:

1. O utilizador preenche os dados no ecrã de registo
2. A conta **não é criada** na BD até que o email seja verificado
3. Os dados ficam pendentes em `SharedPreferences` com expiração de 24h
4. O email contém um código de 6 dígitos (e opcionalmente um deep link `focusvolution://verify-email?token=...`)

Conta de Administrador

As credenciais de administrador estão centralizadas em `AppConfig.kt` :

```
object AppConfig {
    const val ADMIN_USERNAME = "Admin123"
    const val ADMIN_PASSWORD = "Admin2008"
}
```

O login admin é verificado diretamente contra estas constantes, sem passar pela base de dados, garantindo que a conta admin nunca pode ser acidentalmente eliminada.

SMTP Gmail

O envio de emails usa uma **App Password** do Gmail (password de 16 caracteres gerada especificamente para a aplicação), evitando expor a password real da conta Gmail. As credenciais SMTP são configuradas em `FocusVolutionApp.kt`.

Animações

Animação	Localização	Técnica	Duração
Troca de cérebro	MainScreen	Crossfade	400ms
Level-up popup	MainScreen	Animatable bounce + spring	~600ms
Level-down popup	MainScreen	Animatable fade + scale	~500ms
Pulsação timer	MainScreen (≤5s)	infiniteRepeatable + lerp cores	500ms loop
Botões do timer	MainScreen	animateFloatAsState spring scale	Press 0.92x
Transições ecrãs	AppNavigation	slideInHorizontally + fadeIn / slideOutHorizontally + fadeOut	300ms
Itens do histórico	SessionHistoryScreen	AnimatedVisibility com 80ms stagger	300ms cada
Cartões do perfil	ProfileScreen	slideInVertically + fadeIn sequencial (100ms gap)	300ms cada

Navegação

```
Login
├─ (admin) → Admin → AdminUserHistory/{userId}
├─ (registar) → Register → VerifyCode → Login
└─ (user) → Main
    ├─ Settings
    ├─ Profile/{userId} → Chart/{userId}
    └─ SessionHistory/{userId}

Logout → Login
```

Configuração

Email de Verificação (SMTP Gmail)

No ficheiro `FocusVolutionApp.kt` (linhas 42-44), configurar as credenciais Gmail:

```
EmailConfig.smtpUsername = "focusvolution.verifica@gmail.com"
EmailConfig.smtpPassword = "xxxx xxxx xxxx xxxx" // App Password (16 caracteres)
```

```
EmailConfig.fromEmail = "focusvolution.verifica@gmail.com"
```

Para gerar a App Password:

1. Ativar 2FA na conta Gmail: <https://myaccount.google.com/security>
2. Gerar App Password: <https://myaccount.google.com/apppasswords>
3. Copiar a password de 16 caracteres para o campo `smtpPassword`

Credenciais Admin

No ficheiro `app/src/main/java/com/focusvolution/app/config/AppConfig.kt` :

```
object AppConfig {  
    const val ADMIN_USERNAME = "Admin123"  
    const val ADMIN_PASSWORD = "Admin2008"  
}
```

Instalação e Execução

1. Clonar o repositório

```
git clone https://github.com/Rozax15/FocusVolution.git  
cd FocusVolution
```

2. Abrir no Android Studio

- **File > Open...** > seleccionar a pasta do projeto
- Aguardar a sincronização do Gradle (pode demorar 1-2 minutos)
- Se solicitado, confiar no projeto e permitir a sincronização

3. Configurar o email (obrigatório para registo)

Editar `app/src/main/java/com/focusvolution/app/FocusVolutionApp.kt` e colocar as credenciais SMTP (ver secção [Configuração](#)).

Se não configurar o email, o registo de novos utilizadores não funcionará.

4. Executar no emulador ou dispositivo

- Ligar o dispositivo Android via USB (com Debug USB ativado) ou iniciar um emulador
- Clicar em **Run > Run 'app'** (ou Shift+F10)

A aplicação compila e instala automaticamente.

Gerar APK

Via Android Studio

1. **Build > Build Bundle(s) / APK(s) > Build APK(s)**
2. O APK é gerado em: `app/build/outputs/apk/debug/app-debug.apk`

Via linha de comandos (Gradle)

```
# Windows (PowerShell)
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew assembleDebug

# Linux / macOS
./gradlew assembleDebug
```

O APK estará em: `app/build/outputs/apk/debug/app-debug.apk`

Nota: O APK de debug pode ser instalado diretamente no dispositivo.

Funcionalidades

Funcionalidade	Descrição
Temporizador de foco	Define minutos e segundos, inicia/pausa/reinicia, executa em foreground
Barra de progresso	LinearProgressIndicator com pulsação vermelha nos últimos 5s
Sistema de níveis	10 níveis, sobe a cada 5 sessões, desce a cada 3 falhas
Imagens por nível	cerebro_1.png a cerebro_10.png com Crossfade animado
Pop-ups animados	Level-up com bounce, level-down com fade
Efeito de clique	Botões com escala spring 0.92 ao pressionar
Transições entre ecrãs	Slide horizontal + fade com Navigation Compose
Lista animada	Itens do histórico com entrada progressiva (stagger 80ms)
Registo seguro	Password com hash SHA-256, verificação por email (código 6 dígitos)
Histórico de sessões	Lista cronológica com filtro por tags (FilterChip)
Gráfico semanal	Gráfico de barras por dia da semana desenhado com Canvas
Perfil de utilizador	Editar username/password, ver tempo total de foco (com segundos)
Definições	Duração padrão, som (toggle), lembrar último valor
Painel admin	Administrador pode listar todos os utilizadores e ver histórico
Persistência	Dados guardados em SQLite local (Room) com SharedPreferences para definições

Credenciais de Teste

Conta normal

- Registar através do ecrã de registo (necessita email válido para verificação)

Conta admin

- **Username:** Admin123
- **Password:** Admin2008

Utilização da Aplicação

Instalar o APK no Dispositivo

1. Copiar o ficheiro `app-debug.apk` para o dispositivo Android
2. No dispositivo, abrir a pasta onde o APK foi copiado
3. Tocar no ficheiro `app-debug.apk`
4. Se solicitado, permitir a instalação de fontes desconhecidas (apenas esta vez)
5. Clicar em **Instalar**
6. Após a instalação, clicar em **Abrir**

Ecrã de Login

Ao abrir a aplicação, aparece o ecrã de login com duas opções:

- **Entrar** — para utilizadores já registados
- **Registar** — para criar uma conta nova

Criar uma Conta

1. Clicar em **Registar**
2. Preencher:
 - **Username** — nome de utilizador (mín. 3 caracteres)
 - **Email** — email válido (receberá o código de verificação)
 - **Password** — palavra-passe (mín. 6 caracteres)
 - **Confirmar Password** — repetir a password
3. Clicar em **Registar**
4. Verificar a caixa de email — chegará um código de **6 dígitos**
5. Inserir o código no ecrã de verificação
6. Conta ativada! Volta ao ecrã de login automaticamente

Nota: O código expira após 24 horas.

Fazer Login

1. Inserir **email** ou **username**
2. Inserir a **password**
3. Clicar em **Entrar**

Login Admin: Usar `Admin123` / `Admin2008` para aceder ao painel de administração.

Utilizar o Temporizador

Após o login, o ecrã principal mostra:

- **Cérebro:** Imagem que muda conforme o nível (1 a 10)
- **Nível atual:** "Nível X"
- **Sessões concluídas:** "X sessões concluídas" (clicável para ver histórico)
- **Campos de tempo:** Minutos e segundos
- **Barra de progresso:** Indica o tempo restante (pulsa a vermelho nos últimos 5s)
- **Botões:** Iniciar, Pausar, Reiniciar

Iniciar uma sessão:

1. Definir minutos e segundos (ex: 25 minutos)

2. Clicar em **Iniciar** — o timer começa e aparece uma notificação
3. Pode sair da app — o timer continua em background

Durante a sessão:

- **Pausar:** Clicar em **Pausar** para interromper temporariamente
- **Reiniciar:** Clicar em **Reiniciar** para voltar ao tempo inicial
- **Sair da app:** O timer continua, mas ao voltar a sessão é marcada como falhada

Fim da sessão:

- Quando o timer chega a 0, o dispositivo vibra
- A sessão é registada automaticamente no histórico
- Se for a 5.ª sessão concluída, o nível sobe (aparece um popup de parabéns)
- Se saiu da app 3 vezes durante o timer, o nível desce (popup de apoio)

Definições

No ecrã principal, clicar no ícone de **engrenagem** (⚙️) para aceder às definições:

Opção	Descrição
Duração padrão (min)	Valor preenchido automaticamente ao abrir o timer
Som no fim	Ativar/desativar som quando o timer termina
Lembrar último valor	Guardar automaticamente os minutos/segundos da última sessão

Clicar em **Guardar definições** para aplicar as alterações.

Perfil

No ecrã principal, clicar no ícone de **pessoa** (👤) para aceder ao perfil:

- **Informação da conta:** Username e email
- **Tempo total de foco:** Total acumulado de todas as sessões (horas:minutos:segundos)
- **Ver gráfico:** Abre o gráfico de produtividade semanal
- **Alterar username:** Novo nome de utilizador
- **Alterar password:** Password atual + nova password (mín. 6 caracteres)

O gráfico semanal mostra os minutos de foco por dia da semana (Domingo a Sábado). Cada barra representa o total de minutos de foco nesse dia.

Histórico de Sessões

No ecrã principal, clicar no contador de sessões para ver o histórico completo:

- Lista todas as sessões concluídas (data, duração, tag)
- Ordenadas da mais recente para a mais antiga
- Pode filtrar por tag usando os **FilterChip** (ex: "Estudo", "Trabalho")
- Clicar em **Todas** para limpar o filtro

Painel de Administração

Aceder: Fazer login com `Admin123` / `Admin2008` .

Funcionalidades:

- Lista de todos os utilizadores registados, com nível e sessões
- Clicar num utilizador para ver o histórico de sessões
- Clicar no ícone de eliminar para remover um utilizador (com confirmação)

Resolução de Problemas

Problema	Causa	Solução
Não recebo o email de verificação	SMTP não configurado ou credenciais incorretas	Verificar <code>FocusVolutionApp.kt</code> com App Password válida
O timer não aparece após sair da app	Serviço em foreground foi interrompido	Voltar à app — o timer continua em background
A aplicação não abre	Dispositivo Android < 7.0	Atualizar o sistema operativo
Erro ao registar "Username já existe"	Nome de utilizador já registado	Escolher outro username

Licença

Projeto académico — PAP do curso Técnico de Gestão e Programação de Sistemas de Informação (GPSI).

Repositório GitHub: <https://github.com/Rozax15/FocusVolution.git>